

VeroFlow — First Working End-to-End APK Test (With RAG Memory)

What This Version Adds

Previously: screenshot → straight to vision AI → generated code (no memory of past screens).

Now: screenshot → check ChromaDB "have I seen this screen before?" → if yes, reuse the stored label (skip re-labeling) → if no, label it fresh and store it → generate code with that label as extra context.

This is a **basic but real RAG loop** — good enough to show your professor actual progress on the memory/self-healing foundation, without yet having the full validation (Phase 8) or SSIM verification (Phase 9) layered on top.

PART 1: TECH STACK (Updated)

Technology	What It Does In VeroFlow	Status
Python	Main language	✓
FastAPI	Backend API	✓
Ollama + Qwen2.5-VL	Local vision AI — screen labeling + code generation	✓
Appium	Controls the Android emulator	✓
open_clip (CLIP model)	Turns screenshots into 512-number vectors	✓ Now connected
ChromaDB	Stores those vectors + labels, retrieves similar past screens	✓ Now connected
Next.js dashboard	Frontend UI	✓
Jenkins	CI/CD — added later, once core pipeline is stable	✗ Not yet
Validation retry loop (Phase 8)	Re-prompts AI if generated code is broken	✗ Not yet
SSIM verification (Phase 9)	Visually confirms correct screen was reached	✗ Not yet

PART 2: UPDATED FLOW

1. User uploads APK on the dashboard
2. User types instruction: "tap the login button"
3. PHASE 1 (NLP Parser): breaks sentence into structured actions
[{"action": "tap", "target": "login button"}]
4. Backend installs the APK on the emulator (via Appium)
5. Backend takes a screenshot of current screen
6. PHASE 4 (CLIP): converts screenshot into a 512-number vector
7. PHASE 6 (ChromaDB): checks – "have I seen a screen like this before?"
 - If YES (similarity ≥ 0.85): reuse the stored label, skip re-labeling
 - If NO: run PHASE 3 (vision AI labels the screen), then store the new vector + label in ChromaDB for next time
8. Backend sends [structured actions + screen label + screenshot] to Ollama
9. Ollama returns Appium code
10. Backend executes that code on the emulator
11. Backend takes a screenshot AFTER execution
12. Backend returns: Pass/Fail + screen label + whether it was reused or new + screenshots

Note on Phase 5 (filtering): the standalone filter script from before is no longer needed as a separate step — ChromaDB's own similarity search in step 7 already does that exact job (deciding "is this a duplicate or new screen?"). One tool doing double duty, not a missing piece.

PART 3: COMPLETE CODE

File 1: `backend/pipeline/phase6_chromadb_rag.py` (NEW)

```
python
```

```

import chromadb

# This creates a database folder on disk – it remembers data between runs
client = chromadb.PersistentClient(path="storage/chroma_data")

# "cosine" similarity space matches how we compare CLIP vectors elsewhere
collection = client.get_or_create_collection(
    name="screens",
    metadata={"hnsw:space": "cosine"}
)

def find_similar_screen(vector: list, threshold: float = 0.85):
    """
    Searches ChromaDB for a screen similar to the given vector.
    Returns stored screen data if a close match is found, otherwise None.
    """
    if collection.count() == 0:
        return None

    results = collection.query(
        query_embeddings=[vector],
        n_results=1
    )

    if not results['ids'][0]:
        return None

    distance = results['distances'][0][0]
    similarity = 1 - distance # cosine space: similarity = 1 - distance

    if similarity >= threshold:
        metadata = results['metadatas'][0][0]
        return {
            "id": results['ids'][0][0],
            "label": metadata.get('label'),
            "screenshot_path": metadata.get('screenshot_path'),
            "similarity": round(similarity, 3)
        }

    return None

```

```

def store_screen(vector: list, label: str, screenshot_path: str):
    """
    Saves a new screen's vector + label into ChromaDB so it can be
    recognized next time a similar screen appears.
    """
    screen_id = f"screen_{collection.count() + 1}"

    collection.add(
        ids=[screen_id],
        embeddings=[vector],
        metadatas=[{"label": label, "screenshot_path": screenshot_path}]
    )

    return screen_id

if __name__ == "__main__":
    # Quick manual test
    from phase4_clip_encoder import get_image_vector

    vec = get_image_vector("storage/screenshots/screen_test.png")

    existing = find_similar_screen(vec)
    if existing:
        print(f"Found similar screen: {existing}")
    else:
        new_id = store_screen(vec, "Test Screen", "storage/screenshots/screen_test.png")
        print(f"Stored new screen: {new_id}")

```

File 2: `backend/pipeline/test_generator.py` (Updated — now accepts screen label)

```
python
```

```

import ollama
import re
import json

def generate_appium_code(parsed_actions: list, screenshot_path: str, screen_label: str):
    """
    Sends the structured action list + screenshot (+ optional known screen label
    from ChromaDB memory) to the vision AI, and gets back Appium Python code.
    """

    with open(screenshot_path, 'rb') as img_file:
        image_bytes = img_file.read()

    actions_text = json.dumps(parsed_actions, indent=2)

    context_line = ""
    if screen_label:
        context_line = f'\nThis screen has been seen before and was identified as: {screen_label}'

    prompt = f"""
    You are a mobile test automation expert. Look at this Android app screenshot.
    {context_line}
    Perform these actions, in order:
    {actions_text}

    Generate Python code using Appium's "driver" object to perform these actions.
    Rules:
    - Only use: driver.find_element(AppiumBy.XPATH, "...")
    - Use .click() for "tap" actions, .send_keys("text") for "type" actions
    - For "verify" actions, use: assert driver.find_element(...) is not None
    - Return ONLY the Python code, no explanations, no markdown formatting
    - Do not include imports, just the action lines, one line per action

    Example output:
    driver.find_element(AppiumBy.XPATH, "//android.widget.Button[@text='Login']").
    """

    response = ollama.chat(
        model='qwen2.5vl:3b',
        messages=[{
            'role': 'user',

```

```
        'content': prompt,
        'images': [image_bytes]
    }]
)

code = response['message']['content']
code = re.sub(r'```python', '', code)
code = re.sub(r'```', '', code)
code = code.strip()

return code

if __name__ == "__main__":
    sample_actions = [{"action": "tap", "target": "login button"}]
    code = generate_appium_code(sample_actions, "storage/screenshots/screen_test.p
    print(code)
```

File 3: `backend/pipeline/test_executor.py` (Updated — full pipeline connected)

python

```

from appium import webdriver
from appium.options.android import UiAutomator2Options
from appium.webdriver.common.appiumby import AppiumBy
import os
from datetime import datetime

def run_full_test(apk_path: str, instruction: str):
    from pipeline.test_generator import generate_appium_code
    from pipeline.phase1_nlp_parser import parse_instruction
    from pipeline.phase3_screen_labeler import label_screen
    from pipeline.phase4_clip_encoder import get_image_vector
    from pipeline.phase6_chromadb_rag import find_similar_screen, store_screen

    os.makedirs("storage/screenshots", exist_ok=True)
    os.makedirs("storage/chroma_data", exist_ok=True)

    options = UiAutomator2Options()
    options.platform_name = "Android"
    options.device_name = "emulator-5554"
    options.app = apk_path
    options.automation_name = "UiAutomator2"

    driver = webdriver.Remote("http://127.0.0.1:4723", options=options)

    result = {
        "instruction": instruction,
        "parsed_actions": [],
        "screen_label": "",
        "screen_source": "",
        "status": "unknown",
        "generated_code": "",
        "error": None,
        "before_screenshot": "",
        "after_screenshot": ""
    }

    try:
        # PHASE 1: Parse instruction
        parsed_actions = parse_instruction(instruction)
        result["parsed_actions"] = parsed_actions

```

```

if not parsed_actions:
    raise ValueError("NLP parser could not extract any actions from the in

# Screenshot BEFORE action
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
before_path = f"storage/screenshots/before_{timestamp}.png"
driver.get_screenshot_as_file(before_path)
result["before_screenshot"] = before_path

# PHASE 4: Get CLIP vector for this screen
vector = get_image_vector(before_path)

# PHASE 6: Check ChromaDB – have we seen this screen before?
existing_screen = find_similar_screen(vector)

if existing_screen:
    screen_label = existing_screen["label"]
    result["screen_source"] = f"Reused from memory (similarity: {existing_
else:
    # PHASE 3: Not seen before – label it fresh with vision AI
    screen_label = label_screen(before_path)
    store_screen(vector, screen_label, before_path)
    result["screen_source"] = "New screen – labeled and stored"

result["screen_label"] = screen_label

# Generate code, now WITH the screen label as extra context
generated_code = generate_appium_code(parsed_actions, before_path, screen_
result["generated_code"] = generated_code

# Execute the generated code
exec(generated_code, {"driver": driver, "AppiumBy": AppiumBy})

# Screenshot AFTER action
after_path = f"storage/screenshots/after_{timestamp}.png"
driver.get_screenshot_as_file(after_path)
result["after_screenshot"] = after_path

result["status"] = "passed"

except Exception as e:
    result["status"] = "failed"

```

```
        result["error"] = str(e)
    try:
        error_path = f"storage/screenshots/error_{timestamp}.png"
        driver.get_screenshot_as_file(error_path)
        result["after_screenshot"] = error_path
    except:
        pass

    finally:
        driver.quit()

    return result

if __name__ == "__main__":
    result = run_full_test(
        apk_path=r"/path/to/your/app.apk",    # change this
        instruction="tap the login button"
    )
    print(result)
```

File 4: `backend/main.py` (Complete, Updated — Includes Screenshot Serving)

```
python
```

```
from fastapi import FastAPI, UploadFile, File, Form
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
import os
import shutil

from pipeline.phase1_nlp_parser import parse_instruction
from pipeline.test_executor import run_full_test

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

os.makedirs("storage/apks", exist_ok=True)
os.makedirs("storage/screenshots", exist_ok=True)
os.makedirs("storage/chroma_data", exist_ok=True)

@app.get("/")
def home():
    return {"message": "VeroFlow backend is working!"}

@app.post("/upload-apk")
async def upload_apk(file: UploadFile = File(...)):
    save_path = f"storage/apks/{file.filename}"
    with open(save_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    return {"message": "APK uploaded successfully", "path": save_path}

@app.post("/submit-test")
async def submit_test(instruction: str = Form(...)):
    actions = parse_instruction(instruction)
    return {"instruction": instruction, "parsed_actions": actions}
```

```
@app.post("/run-test")
async def run_test(apk_path: str = Form(...), instruction: str = Form(...)):
    result = run_full_test(apk_path, instruction)
    return result

# Serves screenshot images so the Next.js dashboard can display them
app.mount("/screenshots", StaticFiles(directory="storage/screenshots"), name="screenshots")
```

PART 4: COMPLETE SETUP INSTRUCTIONS — TEST FIRST APK

Step 1: Update Requirements

Add these to `backend/requirements.txt` if not already there:

```
chromadb
open_clip_torch
torch
torchvision
```

Install:

```
bash

cd backend
source venv/bin/activate
pip install -r requirements.txt
```

Step 2: Create Folder Structure

```
bash

mkdir -p pipeline storage/apks storage/screenshots storage/chroma_data
```

Step 3: Save All Files

Place each code block above into its matching file path:

- `backend/pipeline/phase6_chromadb_rag.py`
- `backend/pipeline/test_generator.py` (overwrite existing)
- `backend/pipeline/test_executor.py` (overwrite existing)

- `backend/main.py` (overwrite existing)

Make sure `phase1_nlp_parser.py`, `phase3_screen_labeler.py`, `phase4_clip_encoder.py` (from earlier) are also still sitting in `pipeline/` — this new code imports from them.

Step 4: Start Everything (4 Terminals)

Terminal 1 — Emulator: Open Android Studio → Virtual Device Manager → start your emulator, wait for full boot.

Terminal 2 — Appium:

```
bash
appium
```

(leave running)

Terminal 3 — Backend:

```
bash
cd backend
source venv/bin/activate
uvicorn main:app --reload
```

Terminal 4 — Next.js Dashboard:

```
bash
cd veroflow-dashboard
npm run dev
```

Step 5: Confirm Emulator Is Ready

```
bash
adb devices
```

Should show `emulator-5554 device`.

Step 6: Run the Test

1. Open `http://localhost:3000`

2. Upload a **simple** APK (standard buttons/text, not Flutter/game-style)
3. Type a **single-step** instruction: `tap the login button`
4. Click "Run Test"
5. Wait ~15-40 seconds (first run is slower — CLIP model + vision AI both loading)

Step 7: Run It a SECOND Time (This Proves RAG Works)

Run the exact same test again (or a different instruction on the same screen). Check the result — it should now say:

```
"screen_source": "Reused from memory (similarity: 0.9x)"
```

instead of "New screen — labeled and stored". **This is the actual proof for your professor** that the system is remembering screens instead of re-analyzing from scratch every time.

PART 5: WHAT TO HONESTLY TELL YOUR PROFESSOR

What's genuinely working:

- Natural language → structured actions (Phase 1)
- Screenshot → CLIP vector → ChromaDB memory check (Phase 4 + 6)
- New screens get labeled and remembered; repeated screens get recognized instead of re-processed
- AI-generated Appium code executes on a real emulator with before/after screenshots

What's not built yet (say this proactively, don't wait to be asked):

- No validation/retry loop yet if generated code is broken (Phase 8)
- No SSIM visual verification of correct outcome (Phase 9)
- No self-healing on APK version changes yet
- Jenkins CI/CD not yet integrated (planned for after core pipeline is stable)
- Only tested on Android via Appium — Playwright/web scope deferred

This is an honest, defensible "here's a working core loop with real memory, here's what's next" story — which is a much stronger position than overclaiming completeness.